

Intermediate (two-level logger) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Where this exercise is saved: `practice/logging_practice.py` . Run it with `cd practice && python logging_practice.py` .

Used later by: the [Real-world wiring exercise](#) copies the `get_logger()` you write here into `practice/config_logging_wiring/logging_setup.py` , and the [Build Task](#) writes the same function again in `practice/build_task/logging_setup.py` . A copy, not an import — each of those folders stands alone. Keep the name and signature exactly as they are: `get_logger(name: str) -> logging.Logger` .

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

A logger can send its messages to more than one place at once. You can tell it: "show me only the important stuff on screen, but save everything to a file."

Each place it sends messages to is called a **handler**. You'll need two handlers — one for the screen, one for a file — and each handler gets its own cutoff level.

Here are the exact pieces to use:

- `logging.getLogger(__name__)` — get your logger.
- `logging.StreamHandler()` — a handler that prints to the screen.
- `logging.FileHandler("app.log")` — a handler that writes to a file.
- Give each handler its own level with `.setLevel(...)`.
- Attach both handlers to the logger with `.addHandler(...)`.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

A logger can have more than one **handler** attached, and each handler can have its own level. That's the whole trick here: one handler pointed at the screen, set to `INFO` ; one handler pointed at a file, set to `DEBUG` . The logger itself just needs to be set to the *lowest* level of any of its handlers, or messages get filtered out before they even reach the handlers.

Think of it as two filters in a row: the logger's own level is the first gate ("do I even bother with this message?"), and each handler's level is a second gate ("does *this* handler want it?"). A message has to pass both gates to end up somewhere.

Here are the exact pieces, with the calls you'll actually need:

- `logging.getLogger(__name__)` — get your logger.
- `logging.StreamHandler()` — a handler that prints to the screen.
- `logging.FileHandler("app.log")` — a handler that writes to a file.
- `handler.setLevel(logging.INFO)` / `handler.setLevel(logging.DEBUG)` — each handler's own cutoff.
- `logger.addHandler(handler)` — attach a handler (call this twice, once per handler).
- `logger.setLevel(logging.DEBUG)` — the logger's own level must be at least as permissive as its most permissive handler.
- Wrapping all of this in a function named exactly `def get_logger(name: str) -> logging.Logger:` — keep that name, the Real-world wiring exercise and the Build Task both import `get_logger` by it.

Difference between Basic and Intermediate: Basic explains handlers exist, each with its own level, and lists the exact pieces to use. Intermediate explains the two-gate filtering mechanism precisely, gives the exact calls and their order, and introduces wrapping it all in a reusable function — this is the depth the exercise's Solution is written at.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
get a logger
set the logger's own level to the lowest one (DEBUG)

make a screen handler, set it to INFO
make a file handler pointed at "app.log", set it to DEBUG

attach both handlers to the logger

send a DEBUG message -> should show up in the file only
send an INFO message -> should show up on screen AND in the file
send a WARNING message -> should show up on screen AND in the file
```

Here's almost the whole thing — just try running it and reading it line by line:

```
# practice/logging_practice.py
import logging

logger = logging.getLogger(__name__)
logger.setLevel(logging.DEBUG)

screen_handler = logging.StreamHandler()
screen_handler.setLevel(logging.INFO)
logger.addHandler(screen_handler)

# now add a file handler the same way, set to DEBUG
```

Expected output if you run just this: nothing — a handler is attached, but no `.debug()` / `.info()` / `.warning()` call has been made yet, and the file handler is still missing. Add the file handler and the three log calls yourself, then check the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```

define a function get_logger(name: str) -> logging.Logger:
    get a logger named after this module
    set the logger's own level to DEBUG (the lowest, most permissive)

    create a screen handler, set its level to INFO
    create a file handler pointed at "app.log", set its level to DEBUG

    attach both handlers to the logger
    return the logger

logger = get_logger(__name__)

log one DEBUG message    -> should appear in the file only
log one INFO message     -> should appear on screen AND in the file
log one WARNING message  -> should appear on screen AND in the file

```

```

# practice/logging_practice.py
import logging

def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    logger.setLevel(logging.DEBUG)

    screen_handler = logging.StreamHandler()
    screen_handler.setLevel(logging.INFO)
    logger.addHandler(screen_handler)

    # add the file handler yourself, the same way, then return logger

logger = get_logger(__name__)
# add the three log calls yourself – see the Solution if stuck

```

Finish the function, then compare against the [Solution](#).

Difference between Basic and Intermediate: Basic's pseudocode is a flat script that proves the mechanism works — two handlers, two levels — run it once, it works. Intermediate wraps the same steps in a reusable, typed function with a return value.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)